



Nano AI: Technical Debrief

Leveraging **Chrome's Built-in Gemini Nano** for **Offline Inference**.

DEFINITION

Nano AI is a fully offline, browser-based application providing conversational chat and sentiment analysis.

CORE TECH

Powered by Chrome's experimental Prompt API and the built-in Gemini Nano model.



CAPABILITIES

- Conversational Chatbot (Streaming)
- Sentiment Classifier (Structured JSON)
- Full Audit Logging

STATUS

Zero network dependency once cached. Fully local execution.



The Strategic Edge: Local-First Architecture

Traditional Cloud AI

- Privacy: Data leaves device
- Cost: API keys & usage fees
- Latency: Network round-trips
- Availability: Requires Internet

Nano AI (Local)

- Privacy: **100% Local** processing
- Cost: **Zero cost**, no infrastructure
- Latency: **Millisecond** inference
- Availability: **Offline-capable**

Ideal for privacy-sensitive text analysis and edge computing environments.

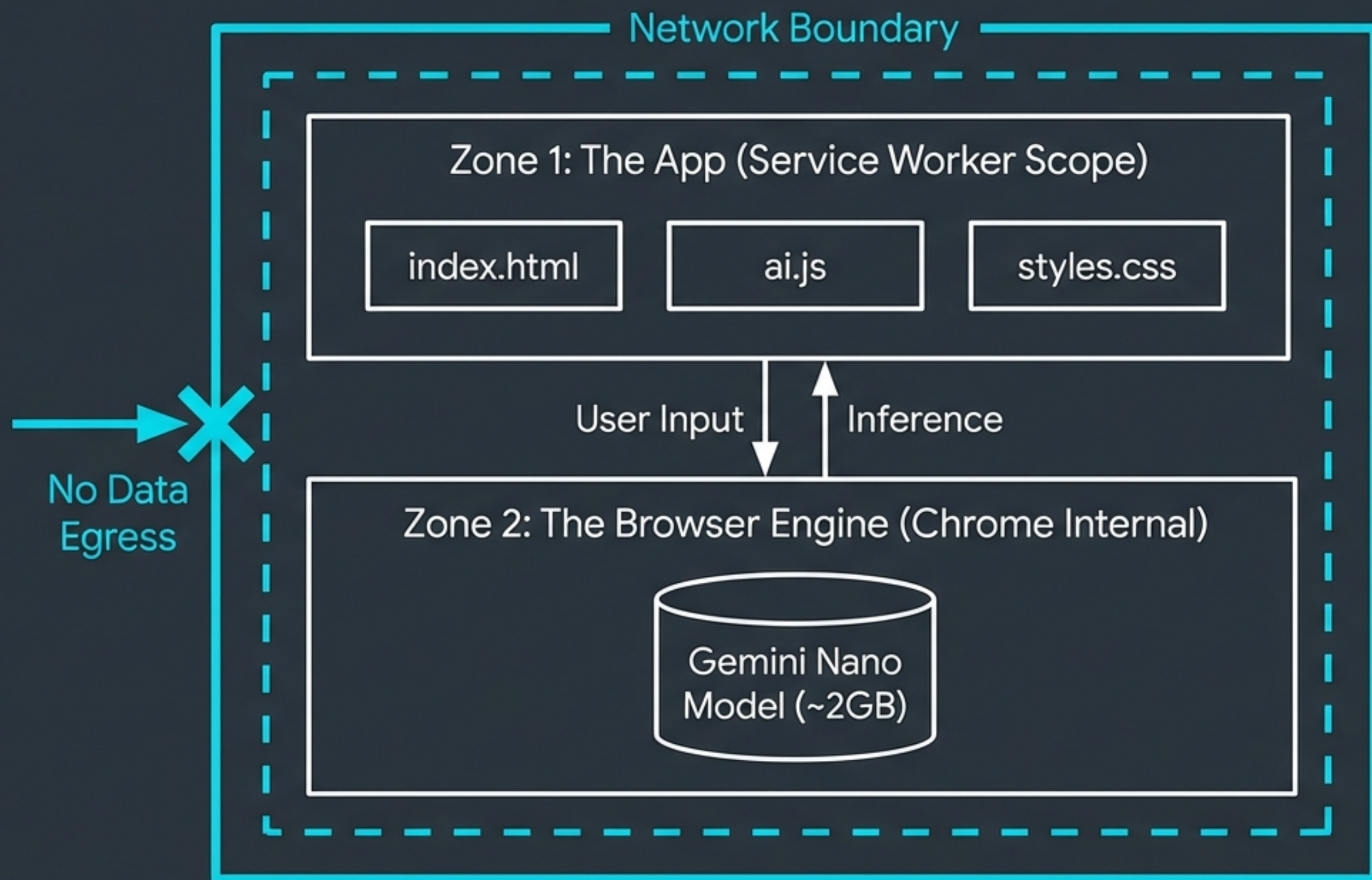
The Engine: Chrome's Prompt API

Direct access via the global 'LanguageModel' class.

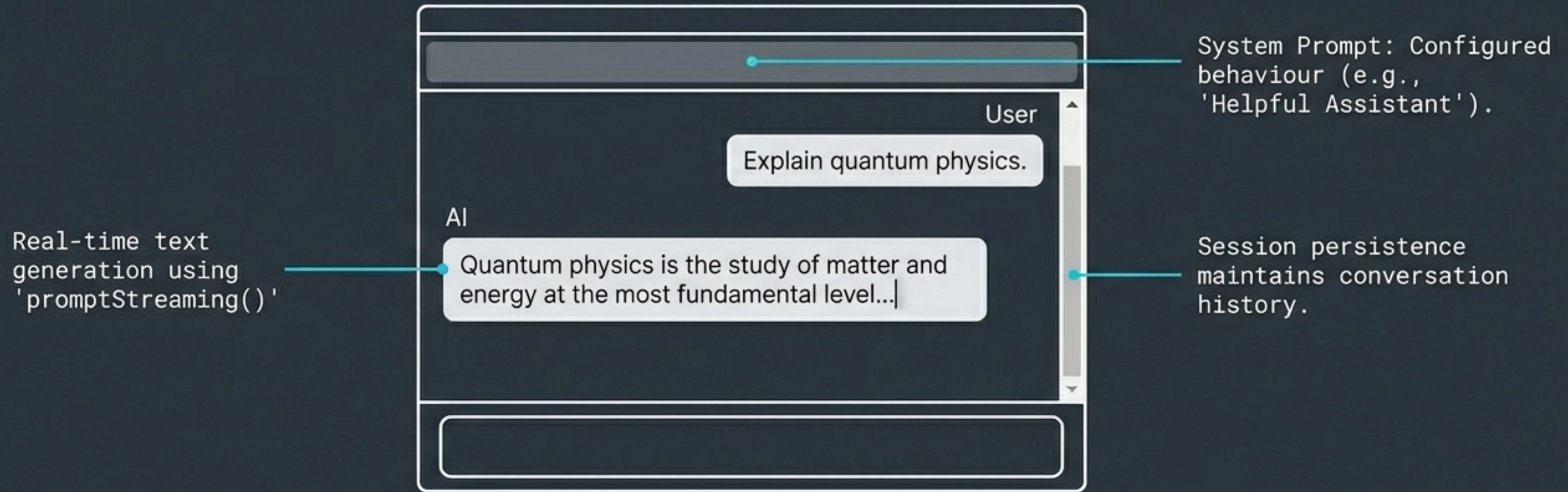
Status	Meaning
available	Model ready for immediate use
downloadable	Model requires download
downloading	Active download state
unavailable	Device incompatible

```
1  const session = await window.ai.languageModel.create({
2    systemPrompt: 'You are a helpful assistant.',
3    temperature: 0.8,
4    topK: 3
5  });
```


Architecture & Data Boundaries



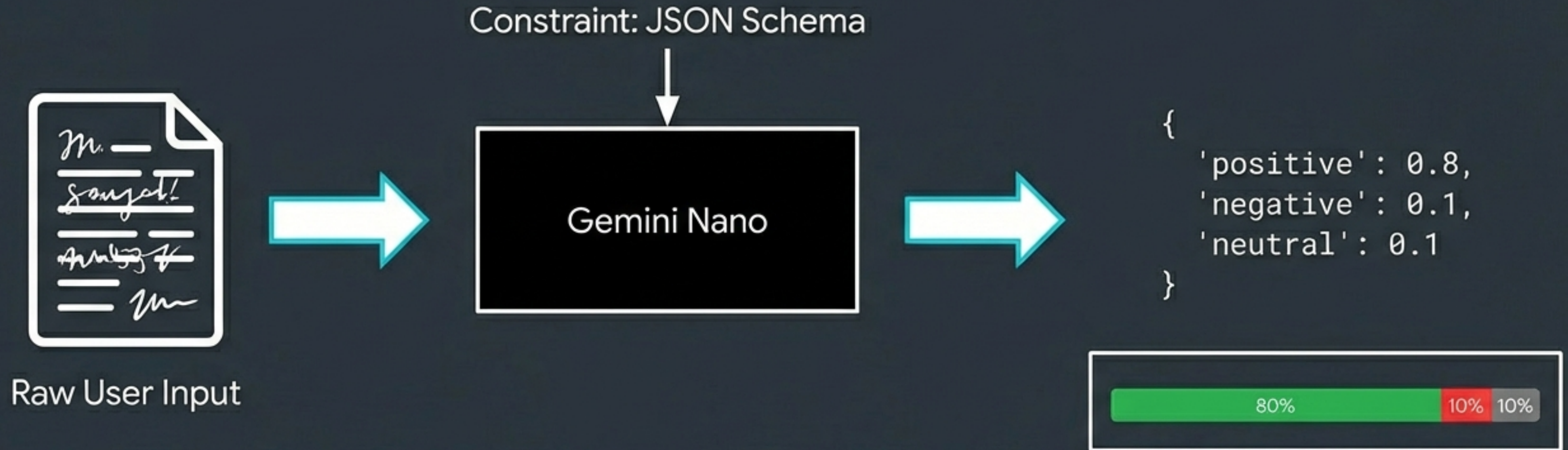
Feature Spotlight: Conversational Chat



Technical Note

Chrome's streaming returns cumulative text (full response), not deltas.

Feature Spotlight: Sentiment Classifier



Utilises JSON Schema to force reliable parsing for multi-class scores.

The Nervous System: Debug & Logging

```
[10:00:01.050] [SYSTEM] Initializing LanguageModel...  
[10:00:05.200] [SENT] User: 'This product is amazing!'  
[10:00:06.800] [RECEIVED] { positive: 0.9, confidence: high }  
[10:00:06.801] [METRICS] Tokens: 45 | Time: 1600ms  
[10:01:00.000] [ERROR] Model download interrupted.
```

Transparency as a feature. A complete audit trail of decision-making.

Code Structure & Design Decisions

```

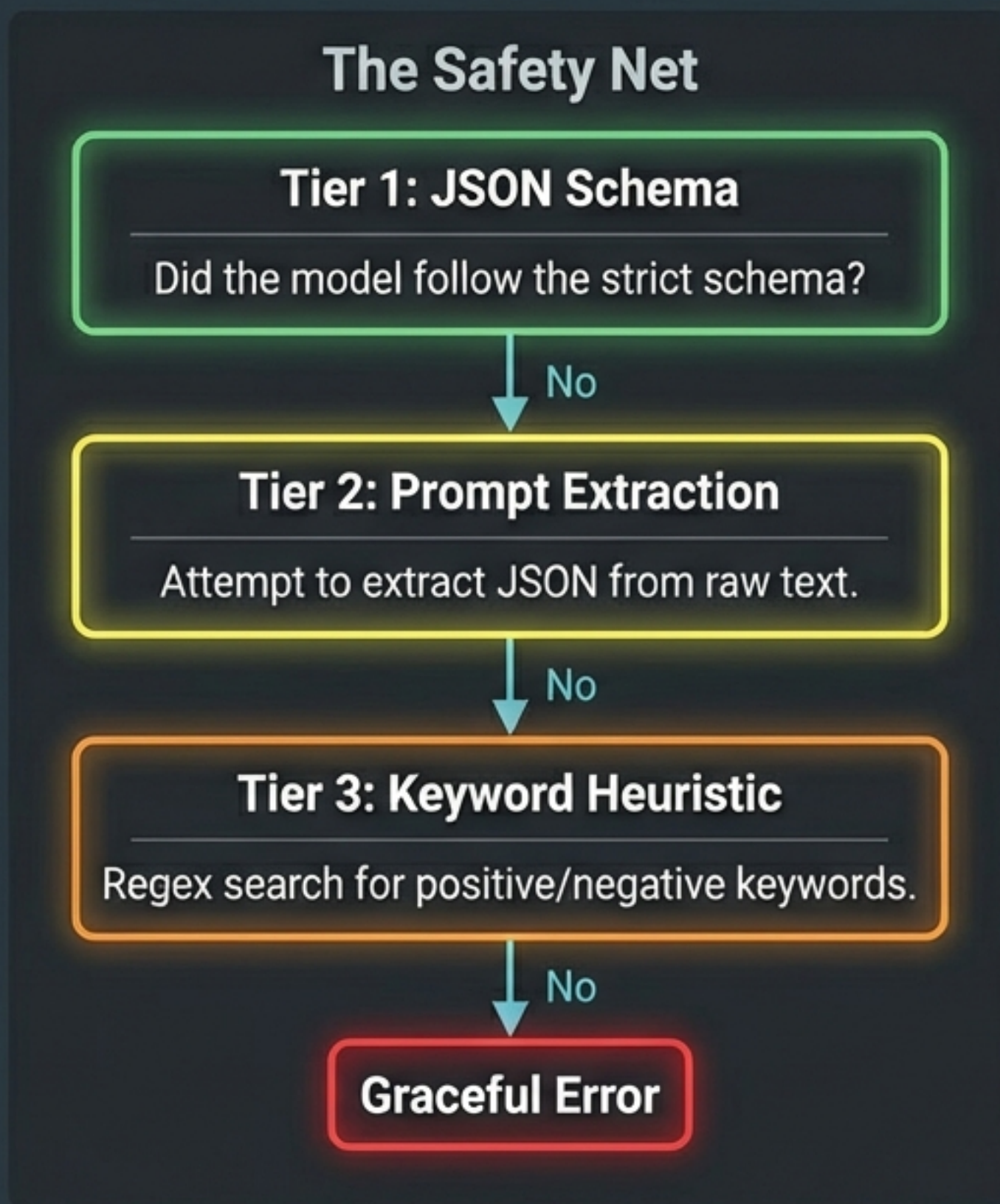
└─ root
  ├── index.html (Layout & Structure)
  ├── styles.css (Theming)
  ├── ai.js (Inference Logic)
  ├── ui.js (DOM Events)
  └── service-worker.js (Caching)

```

Why IIFE over ES Modules?

We use Immediately Invoked Function Expressions (IIFE). ES Modules do not automatically see window globals (like 'LanguageModel') injected by Chrome. Regular scripts avoid this scope injection issue.

Engineering Robustness: The Fallback Chain



Ensures the application never crashes on malformed output.

Achieving True Offline Capability

Application Assets

HTML/CSS/JS via Service Worker (~50KB)

Model Weights

Gemini Nano via Chrome Internal Cache (~2GB)



**TRUE
OFFLINE**

[X] Is the App Cached?

[X] Is the Model Downloaded?

Deployment & Setup Requirements

Hardware Specs

- Storage: 22GB+ free space (Model is ~2GB)
- GPU: 4GB+ VRAM recommended
- CPU: 16GB RAM, 4+ cores (if no GPU)

Software Specs

- Browser: Chrome 138+ (Canary)
- OS: Windows 10/11, macOS 13+, Linux

Critical Chrome Flags

- #optimization-guide-on-device-model: Enabled BypassPerfRequirement
- #prompt-api-for-gemini-nano: Enabled

Current Limitations & Constraints

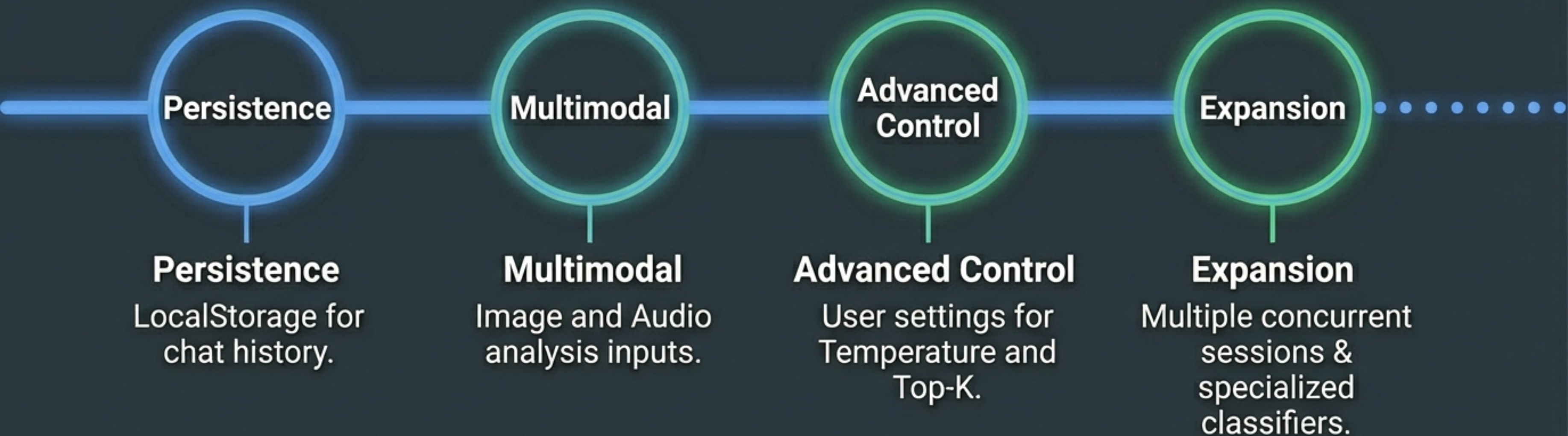
Nano AI is a tool for *processing* text, not necessarily *retrieving* facts.

List of Limitations

1. **Context Window:** ~4,000 tokens. Significantly shorter than cloud models.
2. **Accuracy:** Factual accuracy is not reliable. Prone to hallucinations.
3. **Platform:** Desktop Chrome only. No mobile support yet.
4. **Instruction Following:** Complex prompts may require the fallback strategies.



Future Roadmap



The Paradigm Shift

**“Privacy-preserving AI is no longer
a theoretical concept; it is
becoming a browser primitive.”**

ROBUST • TRANSPARENT • SIMPLE • PRIVATE

